RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS2201-Database management systems

Mini Project Report

Movie Ticket Booking System

Submitted

By

Miruthi Karthikeyan (1RUA25BCA0063)

Niranjana Manjesh (1RUA25BCA0076)

Niveditha M (1RUA25BCA0078)

Shreya B (1RUA25BCA0102)

Under the guidance of:

Professor. Pranjul Shukla

School of Computer Science and Engineering

RV University, Bangalore

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

CERTIFICATE

Certified that the CS2201 – Database management systems, Project work titled *Movie Ticket Booking System* is carried out by *Niveditha.M* who are Bonafide students of the School of Computer Science and Engineering, RV University, Bengaluru, during the year 2026–27. It is certified that all corrections/ suggestions from all the continuous internal evaluations have been incorporated into the project and in this report.

Dr./ Prof. _____

Faculty Guide

Program Director

Abstract

Booking movie tickets by hand can lead to mistakes like selling the same seat twice. Our project is a console based Movie Ticket Booking System written in dbms, which connects to a MySQL database using JDBC. The objectives are to show the movies and the seats available for each show, book tickets, cancel tickets and view all bookings. We designed three tables in MySQL, filled them with sample data and wrote one Java class that runs SQL queries based on the menu choice. The tools used are JDK 17, NetBeans, MySQL Server 8, MySQL Workbench and MySQL Connector/J. The final program updates the available seats correctly after every booking and cancellation, and the data stays saved in the database.

1. Introduction

Movie booking apps keep the movies, shows and seats in a database and update it whenever someone books or cancels. We chose this project to understand how a Java program works with a database, because in class we learnt Java and SQL separately. The same idea is used in bus, train, event and hotel booking, where seats or rooms are limited and the count must always be correct.

2. Scope of the Project

- **Covers:** viewing movies and shows with available seats, booking tickets, cancelling tickets and viewing all bookings.
- **Assumptions:** one user at a time, every show has 50 seats, and MySQL is running on the same computer (port 3306).
- **Limitations:** console only (no GUI), no login or payment, seats are counted but not selected individually, and the queries use string joining instead of PreparedStatement.
- **Applications:** a learning project that can be extended for small cinemas or any event with limited seats.

3. Coding

3.1 Database script (movie_db.sql)

The database movie_db has three tables: movies (title, price), shows (movie, time, total and available seats) and bookings (customer, show, seats, amount). shows and bookings are linked to their parent tables with foreign keys.

movie_db.sql

```sql
CREATE DATABASE movie_db;
USE movie_db;

CREATE TABLE movies (
    movie_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    price DOUBLE NOT NULL
);

CREATE TABLE shows (
    show_id INT PRIMARY KEY AUTO_INCREMENT,
    movie_id INT,
    show_time VARCHAR(50),
    total_seats INT,
    available_seats INT,
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id)
);

CREATE TABLE bookings (
    booking_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100),
    show_id INT,
    seats INT,
    total_amount DOUBLE,
    FOREIGN KEY (show_id) REFERENCES shows(show_id)
);

-- Sample data
INSERT INTO movies (title, price) VALUES
('Inception', 250.0),
('Interstellar', 300.0),
('The Dark Knight', 280.0);

INSERT INTO shows (movie_id, show_time, total_seats, available_seats) VALUES
(1, '10:00 AM', 100, 100),
(1, '06:00 PM', 100, 100),
(2, '02:00 PM', 80, 80),
(3, '09:00 PM', 120, 120);
```

3.2 Java code (MovieTicketBooking.java)

The program loads the MySQL driver, connects using DriverManager.getConnection() and creates a Statement. A menu loop then calls one method for each option. executeQuery() is used for select statements and executeUpdate() for insert, update and delete. While booking,

the seats are checked first, then a row is inserted in bookings and the available seats are reduced. While cancelling, the booking is deleted and the seats are added back.

MovieTicketWeb.java

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.sql.*;
import java.util.HashMap;
import java.util.Map;

public class MovieTicketWeb {

    static final String URL      = "jdbc:mysql://localhost:3306/movie_db";
    static final String USER     = "root";
    static final String PASSWORD = "your_password";   // <-- change this

    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");

        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", new HomeHandler());
        server.createContext("/book", new BookHandler());
        server.createContext("/cancel", new CancelHandler());
        server.createContext("/bookings", new BookingsHandler());
        server.setExecutor(null);
        server.start();

        System.out.println("Server started at http://localhost:8080");
    }

    // ---------- helpers ----------

    static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }

    static Map<String, String> parseForm(String body) {
        Map<String, String> map = new HashMap<>();
        if (body == null || body.isEmpty()) return map;
        for (String pair : body.split("&")) {
            String[] kv = pair.split("=", 2);
            if (kv.length == 2) {
                map.put(URLDecoder.decode(kv[0], StandardCharsets.UTF_8),
                        URLDecoder.decode(kv[1], StandardCharsets.UTF_8));
            }
        }
        return map;
    }

    static void sendHtml(HttpExchange ex, String html) throws IOException {
        byte[] bytes = html.getBytes(StandardCharsets.UTF_8);
        ex.getResponseHeaders().add("Content-Type", "text/html; charset=UTF-8");
        ex.sendResponseHeaders(200, bytes.length);
        try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
    }

    static String layout(String title, String body) {
        return "<!DOCTYPE html><html><head><meta charset='utf-8'><title>" + title + "</title>"
             + "<style>"
             + "body{font-family:Arial;margin:20px;background:#f5f5f5;}"
             + "h1{color:#c0392b;}"
             + "table{border-collapse:collapse;background:#fff;margin:10px 0;}"
             + "th,td{border:1px solid #ccc;padding:8px 14px;}"
             + "th{background:#2c3e50;color:#fff;}"
             + "a.btn,button{background:#27ae60;color:#fff;padding:8px 14px;text-decoration:none;border:none;cursor:pointer;border-radius:4px;}"
             + "input{padding:6px;margin:4px 0;}"
             + ".nav a{margin-right:16px;}"
             + "</style></head><body>"
             + "<h1>🎬 Movie Ticket Booking</h1>"
             + "<div class='nav'><a href='/'>Home</a><a href='/bookings'>All Bookings</a></div><hr>"
             + body
             + "</body></html>";
    }
```

MovieTicketWeb.java

```
    // ---------- handlers ----------

    static class HomeHandler implements HttpHandler {
        public void handle(HttpExchange ex) throws IOException {
            StringBuilder html = new StringBuilder("<h2>Shows Available</h2><table>");
            html.append("<tr><th>Show ID</th><th>Movie</th><th>Time</th><th>Price</th>"
                      + "<th>Total Seats</th><th>Available</th><th>Action</th></tr>");
            try (Connection c = getConnection();
                 Statement s = c.createStatement();
                 ResultSet rs = s.executeQuery(
                    "SELECT s.show_id, m.title, s.show_time, m.price, s.total_seats, s.available_seats " +
                    "FROM movies m JOIN shows s ON m.movie_id = s.movie_id ORDER BY s.show_id")) {
                while (rs.next()) {
                    int id = rs.getInt(1);
                    html.append("<tr>")
                        .append("<td>").append(id).append("</td>")
                        .append("<td>").append(rs.getString(2)).append("</td>")
                        .append("<td>").append(rs.getString(3)).append("</td>")
                        .append("<td>").append(rs.getDouble(4)).append("</td>")
                        .append("<td>").append(rs.getInt(5)).append("</td>")
                        .append("<td>").append(rs.getInt(6)).append("</td>")
                        .append("<td><a class='btn' href='/book?id=").append(id)
                        .append("'>Book</a></td>")
                        .append("</tr>");
                }
            } catch (Exception e) {
                html.append("<tr><td colspan='7'>Error: ").append(e.getMessage()).append("</td></tr>");
            }
            html.append("</table>");
            sendHtml(ex, layout("Home", html.toString()));
        }
    }

    static class BookHandler implements HttpHandler {
        public void handle(HttpExchange ex) throws IOException {
            String method = ex.getRequestMethod();
            String query = ex.getRequestURI().getQuery();
            Map<String, String> params = parseForm(query);

            if ("GET".equalsIgnoreCase(method)) {
                String showId = params.getOrDefault("id", "");
                String body = "<h2>Book Ticket for Show ID " + showId + "</h2>"
                    + "<form method='POST' action='/book'>"
                    + "<input type='hidden' name='show_id' value='" + showId + "'>"
                    + "Name: <input name='name' required><br>"
                    + "Seats: <input name='seats' type='number' min='1' required><br>"
                    + "<button type='submit'>Confirm Booking</button>"
                    + "</form>";
                sendHtml(ex, layout("Book", body));
                return;
            }
```

MovieTicketWeb.java

```
            // POST
            String bodyStr = new String(ex.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
            Map<String, String> form = parseForm(bodyStr);
            String name = form.getOrDefault("name", "").trim();
            int showId = Integer.parseInt(form.get("show_id"));
            int seats = Integer.parseInt(form.get("seats"));

            String message;
            try (Connection c = getConnection()) {
                // Fetch price + available
                PreparedStatement ps = c.prepareStatement(
                    "SELECT m.price, s.available_seats FROM movies m " +
                    "JOIN shows s ON m.movie_id = s.movie_id WHERE s.show_id = ?");
                ps.setInt(1, showId);
                ResultSet rs = ps.executeQuery();
                if (!rs.next()) {
                    message = "❌ Invalid Show ID";
                } else {
                    double price = rs.getDouble(1);
                    int available = rs.getInt(2);
                    if (seats > available) {
                        message = "❌ Only " + available + " seats available";
                    } else {
                        double total = price * seats;
                        PreparedStatement ins = c.prepareStatement(
                            "INSERT INTO bookings(customer_name, show_id, seats, total_amount) VALUES(?,?,?,?)",
                            Statement.RETURN_GENERATED_KEYS);
                        ins.setString(1, name);
                        ins.setInt(2, showId);
                        ins.setInt(3, seats);
                        ins.setDouble(4, total);
                        ins.executeUpdate();
                        ResultSet keys = ins.getGeneratedKeys();
                        int bookingId = keys.next() ? keys.getInt(1) : -1;

                        PreparedStatement upd = c.prepareStatement(
                            "UPDATE shows SET available_seats = available_seats - ? WHERE show_id = ?");
                        upd.setInt(1, seats);
                        upd.setInt(2, showId);
                        upd.executeUpdate();

                        message = "✅ Booking ID <b>" + bookingId + "</b> confirmed!<br>"
                                + "Seats: " + seats + "<br>Total: Rs. " + total;
                    }
                }
            } catch (Exception e) {
                message = "❌ Error: " + e.getMessage();
            }
            String body = "<h2>Booking Result</h2><p>" + message + "</p>"
                        + "<a class='btn' href='/'>Back to Home</a>";
            sendHtml(ex, layout("Result", body));
        }
    }

    static class BookingsHandler implements HttpHandler {
        public void handle(HttpExchange ex) throws IOException {
            StringBuilder html = new StringBuilder("<h2>All Bookings</h2><table>");
            html.append("<tr><th>ID</th><th>Customer</th><th>Movie</th>"
                      + "<th>Time</th><th>Seats</th><th>Total</th><th>Action</th></tr>");
            try (Connection c = getConnection();
                 Statement s = c.createStatement();
                 ResultSet rs = s.executeQuery(
                    "SELECT b.booking_id, b.customer_name, m.title, s.show_time, b.seats, b.total_amount " +
                    "FROM bookings b JOIN shows s ON b.show_id = s.show_id " +
                    "JOIN movies m ON s.movie_id = m.movie_id ORDER BY b.booking_id")) {
                boolean any = false;
                while (rs.next()) {
                    any = true;
                    int id = rs.getInt(1);
                    html.append("<tr>")
                        .append("<td>").append(id).append("</td>")
                        .append("<td>").append(rs.getString(2)).append("</td>")
                        .append("<td>").append(rs.getString(3)).append("</td>")
                        .append("<td>").append(rs.getString(4)).append("</td>")
                        .append("<td>").append(rs.getInt(5)).append("</td>")
                        .append("<td>").append(rs.getDouble(6)).append("</td>")
                        .append("<td><a class='btn' href='/cancel?id=").append(id)
                        .append("'>Cancel</a></td></tr>");
                }
                if (!any) html.append("<tr><td colspan='7'>No bookings yet</td></tr>");
            } catch (Exception e) {
                html.append("<tr><td colspan='7'>Error: ").append(e.getMessage()).append("</td></tr>");
            }
            html.append("</table><br><a class='btn' href='/'>Back to Home</a>");
            sendHtml(ex, layout("Bookings", html.toString()));
        }
    }
```

MovieTicketWeb.java

```
217      static class CancelHandler implements HttpHandler {
218          public void handle(HttpExchange ex) throws IOException {
219              Map<String, String> params = parseForm(ex.getRequestURI().getQuery());
220              int bookingId = Integer.parseInt(params.get("id"));
221              String message;
222              try (Connection c = getConnection()) {
223                  PreparedStatement ps = c.prepareStatement(
224                      "SELECT show_id, seats FROM bookings WHERE booking_id = ?");
225                  ps.setInt(1, bookingId);
226                  ResultSet rs = ps.executeQuery();
227                  if (!rs.next()) {
228                      message = "❌ Booking ID not found";
229                  } else {
230                      int showId = rs.getInt(1);
231                      int seats = rs.getInt(2);
232
233                      PreparedStatement del = c.prepareStatement(
234                          "DELETE FROM bookings WHERE booking_id = ?");
235                      del.setInt(1, bookingId);
236                      del.executeUpdate();
237
238                      PreparedStatement upd = c.prepareStatement(
239                          "UPDATE shows SET available_seats = available_seats + ? WHERE show_id = ?");
240                      upd.setInt(1, seats);
241                      upd.setInt(2, showId);
242                      upd.executeUpdate();
243
244                      message = "✅ Booking cancelled. " + seats + " seat(s) freed.";
245                  }
246              } catch (Exception e) {
247                  message = "❌ Error: " + e.getMessage();
248              }
249              String body = "<h2>Cancel Result</h2><p>" + message + "</p>"
250                          + "<a class='btn' href='/bookings'>Back to Bookings</a>";
251              sendHtml(ex, layout("Cancel", body));
252          }
253      }
254  }
```

4. Screenshots

```
10  import com.sun.net.httpserver.HttpExchange;
11  import com.sun.net.httpserver.HttpHandler;
12  import com.sun.net.httpserver.HttpServer;
13
14  import java.io.IOException;
15  import java.io.OutputStream;
16  import java.net.InetSocketAddress;
17  import java.net.URLDecoder;
18  import java.nio.charset.StandardCharsets;
19  import java.sql.*;
20  import java.util.HashMap;
21  import java.util.Map;
22
23  public class MovieTicketBooking {
24
25      static final String URL      = "jdbc:mysql://localhost:3306/movie_db";
26      static final String USER     = "root";
27      static final String PASSWORD = "nira";   // <-- change this
28
29      public static void main(String[] args) throws Exception {
30          Class.forName("com.mysql.cj.jdbc.Driver");
31
32          HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
33          server.createContext("/", new HomeHandler());
34          server.createContext("/book", new BookHandler());
35          server.createContext("/cancel", new CancelHandler());
36          server.createContext("/bookings", new BookingsHandler());
37          server.setExecutor(null);
38          server.start();
39
40          System.out.println("Server started at http://localhost:8080");
41      }
```

Figure 1: Database connection code in NetBeans

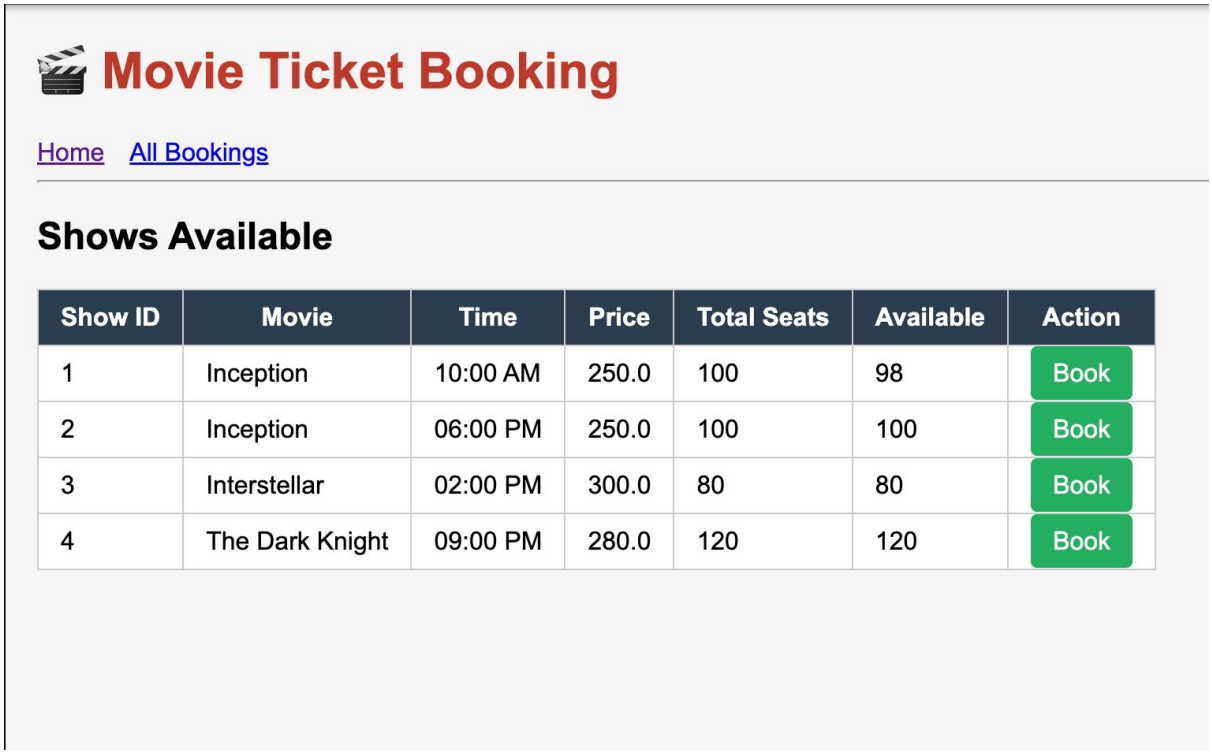


Figure 2: Program output (connected successfully, movie list, menu and booking)

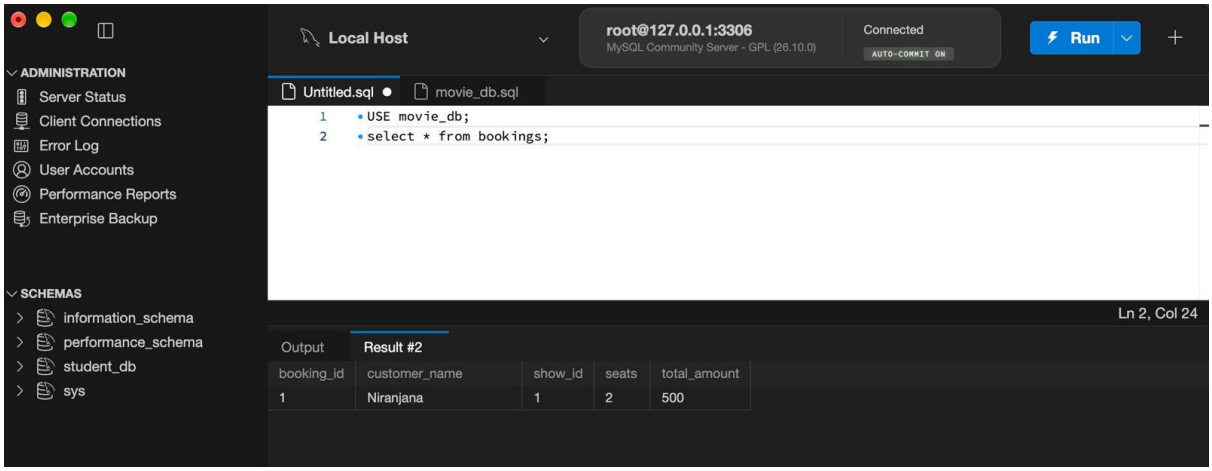


Figure 3: MySQL Workbench connected to Local Host and the bookings table

5. Conclusion

We made a Movie Ticket Booking System using Java, JDBC and MySQL. It can show movies and free seats, book and cancel tickets, and list all bookings. In our testing the seat count went down after a booking and came back after a cancellation, and the total amount was correct. The project helped us understand how the driver, connection, Statement and ResultSet work

together, and how a small mistake like a missing jar or a wrong password can stop the program. In future we would like to add a GUI, a login and PreparedStatement.

6. AI Tool Usage Report

- **AI tool used:** Claude (Anthropic).
- **Purpose:** To get basic guidance on Java JDBC connectivity, understand the initial setup requirements, and troubleshoot connection errors between NetBeans and MySQL Workbench.
- **Sample prompt:** "I am a beginner working on a college JDBC project called Movie Ticket Booking System (Java in NetBeans, MySQL in Workbench). Please explain the basic steps required to establish a connection between my Java project and MySQL database."
- **Generated outputs:** Basic JDBC connection guidance, suggestions for handling database operations, and troubleshooting steps.
- **Modifications by us:** We independently worked on the project setup, explored the required browser URLs and resources, configured the database, added the MySQL Connector/J jar, modified the connection details, and implemented and tested the booking, cancellation, and viewing functionalities with different shows. We also captured the screenshots and prepared the conclusion ourselves.
- **Contribution of the AI tool:** Claude was used mainly as a reference for understanding JDBC connectivity and resolving a few technical issues. The project setup, implementation, testing, and final modifications were carried out by our team.